IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 22 October 2024, accepted 13 November 2024, date of publication 20 November 2024, date of current version 3 December 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3502665

RESEARCH ARTICLE

ON-CNN: Low Latency and High Throughput Online Arithmetic-Based Convolutional Neural Network Accelerator

MUHAMMAD AKMAL SHAFIQUE, (Graduate Student Member, IEEE), AND JEONG-A LEE, (Life Senior Member, IEEE)
Department of Computer Engineering, Chosun University, Gwangju 61452, Republic of Korea

Corresponding author: Jeong-A Lee (jalee@chosun.ac.kr)

This work was supported in part by the Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education under Grant NRF-2020R1I1A3063857.

ABSTRACT Online arithmetic also known as most-significant-digit-first (MSDF) computing is becoming a choice for hardware implementation of different machine learning algorithms. It enables the generation of MSD as the first output digit and usage in consecutive computations in digit serial manner leading to different benefits like low latency, high throughput, variable precision and early termination etc. Recently, the demand for real time implementations of convolutional neural network (CNN) accelerator has dramatically increased. Latency and throughput of the CNN accelerator have paramount importance in real time object detection in video/image classification tasks. In this paper, we present online arithmetic based CNN accelerator architecture (ON-CNN), which exploits the computational capabilities of online arithmetic to achieve lowest latency and highest throughput without affecting accuracy. It also employs a re-configurable power saving strategy which saves significant percentage of power in execution of different convolution layers. ON-CNN has two versions: one whose processing element (PE) takes input in serial-serial manner (ON-CNN$_{SS}$) and second whose PE takes input in serial-parallel manner (ON-CNN$_{SP}$). ON-CNN$_{SS}$ achieves speed-up in execution time from 145.6 to 5.7 times in comparison with state-of-the-art CNN accelerator architectures. Whereas, ON-CNN$_{SP}$ achieves speed-up in execution time from 162.0 to 6.4 times in comparison with state-of-the-art CNN accelerator architectures. ON-CNN$_{SS}$ and ON-CNN$_{SP}$ show the highest peak-throughput of 2.276 TOPS and 2.535 TOPS respectively for VGG-16 and ResNet-50 model. The proposed re-configurable power saving strategy saves overall 19.2%, 33.8% and 18.9% power while executing VGG-16, ResNet-18 and ResNet-50 models, respectively. Comparison using metrics like area efficiency (TOPS/mm^2) and power efficiency (TOPS/W) shows that ON-CNN$_{SS}$ and ON-CNN$_{SP}$ perform better than state-of-the-art CNN accelerator architectures.

INDEX TERMS Low latency, high throughput, online arithmetic, MSDF computing, power saving, convolutional neural network, hardware accelerator.

I. INTRODUCTION

Deep neural networks (DNNs) are fundamental in various tasks such as classification, regression, and pattern recognition [1]. They are capable of learning complex hierarchical representations from data, which makes them suitable for tasks with high-dimensional input spaces. DNNs have been bringing revolution in areas like natural language processing, computer vision, and speech recognition etc. Convolutional neural networks (CNNs), are a specialized form of DNNs, which are particularly powerful for processing structured grid-like data, such as images [2]. They leverage the concept of local connectivity and shared weights to efficiently learn spatial hierarchies of features. CNNs have significantly

The associate editor coordinating the review of this manuscript and approving it for publication was Frederico Guimarães.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

advanced the field of computer vision, enabling tasks like image classification [3], [4], object detection, and semantic segmentation [5] with remarkable accuracy. Until now, graphical processing units (GPUs) have been employed for CNN acceleration. However, they exhibit high latency and power consumption. This has pushed researchers to exploit application-specific-integrated-circuits (ASICs) which offer low latency, high throughput, small area and less power consumption. Thus, the demand for ASICs based hardware accelerators for CNNs is increasing.

Researchers have employed different computing paradigms to lower the latency and to increase the throughput of CNN accelerators. Bit-parallel computing have been widely researched in this domain [6], [7], [8], [9], [10], [11]. It processes/generates the input/output in parallel form. Recently bit-serial computing has emerged as a potential candidate for future CNN accelerators [12], [13]. Bit-serial computing techniques process the input in serial manner, but generates the output in parallel form. Bit-serial processing element (PE) provides low latency, small area and less power consumption in comparison with bit-parallel PE. However, it exhibits low throughput in comparison with bit-parallel PE. In order to match the throughput, bit-serial CNN accelerators employ more number of PEs. Thus, increasing overall area and power consumption.

We propose to employ online arithmetic based bit-serial computing which offers lower latency and higher throughput in comparison with conventional bit-serial computing. Online arithmetic computing is a form of bit-serial computing which not only processes the input in online manner i.e. most-significant-digit-first (MSDF) form, but also generates the output in online manner [14]. Conventional bit-serial computing techniques process the input in least-significant-digit-first (LSDF) form, but generates the output in parallel form. Thus, the next arithmetic unit has to wait for the parallel output. Whereas, online arithmetic units can take input and generate output in online form. Thus, online arithmetic based computing can serve as a potential candidate to address the bottlenecks of conventional computing paradigms. It achieves significant improvement in latency and throughput. Hence, achieving better trade-off between area and power efficiency [15]. Like conventional bit-serial computing [12], [13], online arithmetic computing also allows variable numerical precision (VNP) for both weight/filter and activation.

MSDF-SGD [16] presents the application of online arithmetic for stochastic gradient descent (SGD). It enables the low-precision implementation of the SGD algorithm which is required for improvement in throughput and efficient convergence. MSDF-SGD attains 16x speed up and converges up to 8.6x faster as compared to state-of-the-art low-precision hardware implementations. MSBF-LSTM [17] presents novel most-significant-bit-first (MSBF) based implementation of long short-term memory (LSTM) network. It employs incremental network quantization to achieve high accuracy with low-precision weights. In [18] the authors propose quantum MSDF addition for quantum computing and shows the advantages over quantum carry propagate addition. They also present a VNP method for quantum computing. FGIE [19] presents an online arithmetic based neural inference engine. It features early termination of ineffectual computations and VNP processing. It focuses on the acceleration of fully-connected (FC) layers and does not consider convolution layers. ON-CNN proposes a better way for online processing using online multipliers and adders. It also deals with convolution layers.

In order to have low latency, high throughput and low power consumption in CNN accelerators, we propose an online arithmetic based hardware accelerator for CNN computation (ON-CNN). It employs online arithmetic units. These units are arranged in cascade form in such a way that next unit starts computation as soon as it receives the MSD output of the previous unit. This approach significantly improves the latency and throughput of the PE. We have also proposed a re-configurable power saving strategy to reduce the power consumption of the proposed ON-CNN accelerator based on the characteristics of CNN layers. The main contributions of this paper are outlined as follows:

- We propose a novel online arithmetic based hardware accelerator (ON-CNN) for CNN computation
- We propose two versions of ON-CNN: one which takes input in serial-serial manner (ON-CNN$_{SS}$) and second which takes input in serial-parallel manner (ON-CNN$_{SP}$)
- We have employed an efficient data-flow that can process filters of any dimension
- We propose an online execution approach based on online arithmetic and formulate the framework for calculation of computation cycles
- We propose a re-configurable power saving strategy which switches off idle resources to reduce power consumption
- We evaluate ON-CNN against state-of-the-art CNN accelerators [6], [7], [8], [9], [10], [11], [12], [13]
- We have shown that ON-CNN improves the execution time of CNN layers and achieves high throughput in terms of TOPS in comparison with state-of-the-art CNN accelerator architectures

The rest of the paper is organized as follows; Section II presents the related work. Section III presents the overview of online arithmetic. Section IV presents the baseline data-flow, proposed online-execution approach, proposed ON-CNN accelerator architecture, and proposed re-configurable power saving strategy. Section V presents the result & analysis section. Section VI concludes the paper.

II. RELATED WORK

The brief overview and comparative analysis of each architecture which we have used for comparison is as follows:

Eyeriss [6] is a CNN accelerator which focuses on energy-efficient CNN computation. It improves energy-efficiency by reducing movement of data and making use

of data statistics i.e. eliminating zero valued data which causes ineffectual computations. It leverages the proposed row-stationary data-flow for reducing energy consumption due to memory accesses. It is a fixed 16-bit design. The arithmetic data-path of PE consists of conventional two-stage pipelined multiplier and adder. It executes all layers of VGG-16 [4] in 4309.5 ms and delivers the peak throughput of 0.0214 TOPS. Res-DNN [9] is another CNN accelerator based on Eyeriss. Unlike Eyeriss which employs binary number system (BNS), it employs residue number system (ResNS). It reduces the energy consumption by employing energy-efficient implementation of multipliers and adders in ResNS. It also realizes post-processing units (PPU) like activation and pooling in ResNS. Res-DNN doesn't provide the execution time for any CNN model. For VGG-16, it delivers the peak throughput of 0.134 TOPS which is higher than 0.0214 TOPS peak throughput of Eyeriss.

Stripes [12] is a bit-serial computing based CNN accelerator which focuses on VNP processing. The PE of Stripes employs serial-parallel shift-and-add multipliers. It takes one input in serial form i.e. IFM window (activations) and other input in parallel form i.e. filters (weights). It is a fixed 16-bit architecture. It executes all layers of VGG-16 in 38.1 ms and delivers the peak throughput of 1.260 TOPS. Loom [13] is another bit-serial computing based CNN accelerator which focuses on VNP processing. The PE of Loom employs serial-serial shift-and-add multipliers. It takes both inputs in serial manner. It is a fixed 16-bit architecture. It executes all layers of VGG-16 in 515.9 ms and delivers the peak throughput of 0.093 TOPS (We have implemented and synthesized, Stripes and Loom using 45 nm technology library. The input bit precision is 16-bit and accumulator width is set to be 64-bit, same as of proposed ON-CNN accelerator architecture).

FID (Fully-connected Inspired Data-flow) [7] is a CNN accelerator which aims to reduce the execution time for computing convolution layers. It proposes a data-flow to process CNN workload as FC workload. It has two design versions: first is area-efficient version which we will call FID(A-E) and second is memory access efficient version which we will call FID(MA-E). Both are fixed 16-bit designs. The arithmetic data-path of PE consists of conventional multiplier and adder. FID(A-E) executes all layers of VGG-16 in 436.4 ms and delivers the peak throughput of 0.0703 TOPS. Whereas, FID(MA-E) executes all layers of VGG-16 in 453.3 ms and delivers the peak throughput of 0.0677 TOPS. In [11] authors propose a CNN accelerator based on FID, which aims to reduce the execution time for computing convolution layers. We will call it BLMAC. Like FID, it focuses only on acceleration of CNN having 3×3 filter dimension. The main contribution is to minimize the critical path delay by employing logic level design optimizations like factoring out carry propagate adders (CPAs) by using carry save adders (CSAs) and employing higher radix multiplier for implementation of a PE. It is a fixed 16-bit architecture. It executes all layers of VGG-16 in 169.1 ms and delivers the peak throughput of 0.181 TOPS.

ZASCA (Zero Activation Skipping Convolution Accelerator) [8] is another CNN accelerator inspired by FID. FID focuses on acceleration of CNN having 3×3 filter dimension. ZASCA deals with filters with varying dimensions. It is a re-configurable architecture. It has two versions: ZASCAD which consider dense representation of activation and ZASCAS which considers sparse representation of activation. The arithmetic data-path of PE consists of conventional multiplier and adder. It is a fixed 16-bit architecture. ZASCAD executes all layers of VGG-16 in 421.8 ms and delivers the peak throughput of 0.0725 TOPS. Whereas, ZASCAS executes all layers of VGG-16 in 246.8 ms and delivers the peak throughput of 0.1239 TOPS. CARLA [10] is another CNN accelerator which focuses on energy-efficient CNN computation. It aims to improve the utilization of PEs. Unlike FID and like ZASCA, it deals with filters of different dimensions. It is able to process filters of different dimension because of proposed data-flow and re-configuration. It is a fixed 16-bit architecture. It executes all layers of VGG-16 in 396.9 ms and delivers the peak throughput of 0.0774 TOPS.

In the previously discussed state-of-the-art CNN accelerator architectures there are three research challenges that we have identified. First is the need for architecture that can process filters of any dimension. Architectures employ re-configuration and multiple data-flows to process each filter dimension. This solution increases the area overhead due to re-configuration circuitry. And still unable to deal efficiently with filters of any dimension. Second research challenge is to reduce the execution time for computation of a CNN model. Architectures employ efficient data-flows to increase resource utilization, which results in reduction in computation cycles leading to reduction in execution time. Another approach is to use logic-level optimizations for computational logic i.e. multipliers and adders to reduce the critical-path delay. It results in increase in frequency and reduction in execution time. Third research challenge is to design a power-efficient accelerator. Architectures employ small number of PEs, energy-efficient data-flows which reduce data movement, different number system representation etc. These techniques limit the peak throughput of the architecture.

In this work, we propose a CNN accelerator architecture which addresses all these three research challenges. In order to address the first research challenge, we employ a data-flow which can process filter of any dimension. This data-flow works in such a way that architecture of a PE does not limit the the dimension of filter it can process. In order to address the second research challenge, we propose to employ online-arithmetic based computing to accelerate the computational speed. It results in significant reduction in execution time and increase in peak throughput. In order to address the third research challenge, we propose a re-configurable power saving strategy which switch-off idle PEs based on the characteristic of a CNN layer. It significantly reduces the power consumption for different CNN layers.

III. PRELIMINARIES

A. OVERVIEW OF ONLINE ARITHMETIC

Online arithmetic or MSDF computing (left-to-right (LR) arithmetic) [20] is a computing paradigm which can overcome the limitations of conventional arithmetic (right-to-left (RL) arithmetic). It processes the input and generates the output in digit serial form (Here we are using the term digit instead of bit because online arithmetic units process the number in redundant number system (RNS). Thus, a digit consists of two bits). Online arithmetic unit works in iterations. In each iteration, it takes an input digit starting from MSD and generates an output digit starting from MSD. There is delay of δ cycles (known as online delay) between the application of MSD input and generation of MSD output. During these δ cycles, online arithmetic unit receives enough input digits to be able to compute the MSD output. δ is a small number and is different for different arithmetic operations [14]. Online arithmetic unit requires redundant number representation in computing the output digits based on partial input. RNS provides several representations of a given number. By using RNS representation for the output digits, if necessary, online arithmetic unit can introduce compensation in the following iterations to form the correct final output.

For sequence of dependable arithmetic operations (Addition, multiplication, division, square root etc.) there is timing difference in processing of input and generation of output between online and conventional arithmetic computing. Fig. 1 shows this phenomenon for three dependable arithmetic operations having different δ. In case of online arithmetic, for sequence of dependable arithmetic operations, the next operation starts once for the previous operation the MSD output digits has generated. On the contrary, in conventional arithmetic the next operation starts once the previous operation has generated the whole output. Thus, for sequence of dependable arithmetic operations online arithmetic produces the final output earlier in comparison with conventional arithmetic. Now, we will briefly discuss the working of online arithmetic units employed by ON-CNN i.e. online multiplier, online adder and on-the-fly converter.

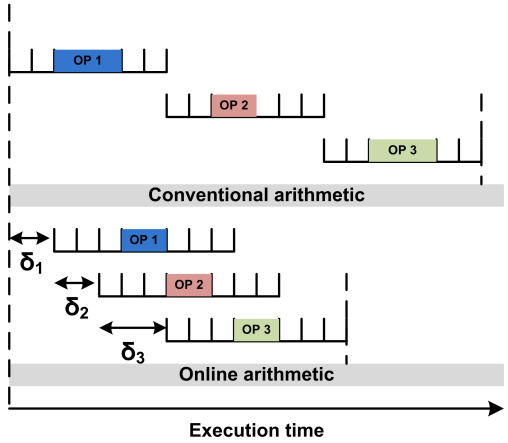

FIGURE 1. Execution time comparison of conventional and online arithmetic for sequence of three dependable arithmetic operations (OP 1, OP 2, OP 3).

Algorithm 1 Radix-2 OSSM Algorithm

procedure $OSSM(X, Y, P_{out})$
X: Serial input (Element of input feature map)
Y: Serial input (Element of filter)
P_{out}: Serial output
1. [Initialize]
  $x[-3] = y[-3] = w[-3] = 0$
  for $(j = -3, -2, -1)$ **do**
    $X[j+1] = CA - REGX(X[j], x_{j+4})$
    $Y[j+1] = CA - REGY(Y[j], y_{j+4})$
    $v[j] = 2w[j] + (X[j].y_{j+4} + Y[j+1].x_{j+4})2^{-3}$
    $w[j+1] \leftarrow v[j]$
  end for
2. [Recurrence]
  for $(j = 0, \ldots, n-1)$ **do**
    $v[j] = 2w[j] + (X[j].y_{j+4} + Y[j+1].x_{j+4})2^{-3}$
    $p_{j+1} = SELM(\widehat{V}[j])$
    $w[j+1] \leftarrow v[j] - p_{j+1}$
    $P_{out} \leftarrow p_{j+1}$
  end for
end procedure

1) ONLINE SERIAL-SERIAL MULTIPLIER (OSSM)

Radix-2 OSSM is depicted in Fig. 2 (a) [15]. It takes both inputs (X and Y) in serial manner. δ for OSSM (δ_M) is 3. It follows the process of Algorithm 1 [15]. For n-digit input, it consists of $n + \delta_M$ iterations. One digit of both serial inputs (X and Y) is introduced during cycles -δ_M to $n - 1 - \delta_M$. After that input is kept 0 for rest of cycles. Initially the output (P_{out}) is 0 for cycles -δ_M to -1. After that one digit of output (P_{out}) is produced in each cycle starting from cycle 0. The derivation of the algorithm for OSSM is presented in [22]. OSSM processes the partial products internally in carry-save form (W_s and W_c) and produces the output (P_{out}) in signed-digit form in the range of $(-1, 1)$.

As seen in Fig. 2 (a), OSSM works such that both the inputs X and Y come in MSDF form. The CA-REG consists of on-the-fly converter (OFC) which converts the input from RNS to BNS representation. Then, the SELECTOR chooses the partial product to be added based on the input digit. If the input digit is 0 then the SELECTOR outputs 0, 1 then the SELECTOR outputs the output of CA-REG and -1 then the SELECTOR outputs the inverted output of CA-REG. The CSA 4:2 adds the partial products with the residual stored in REG W_S and REG W_C. Here the input carries C_X and C_Y correspond to the sign of the MSDF input digit X and Y, respectively. The V module consist of a 4-bit CPA. It produces the estimate $\widehat{V}$ i.e. $(v_{-1}v_0v_1v_2)$ by adding the first 4-bit of the carry save format output of CSA 4:2. SELECTION FUNCTION (SELM) is the product-digit selection function which selects the output digit based on Eq. 1. Using the fact

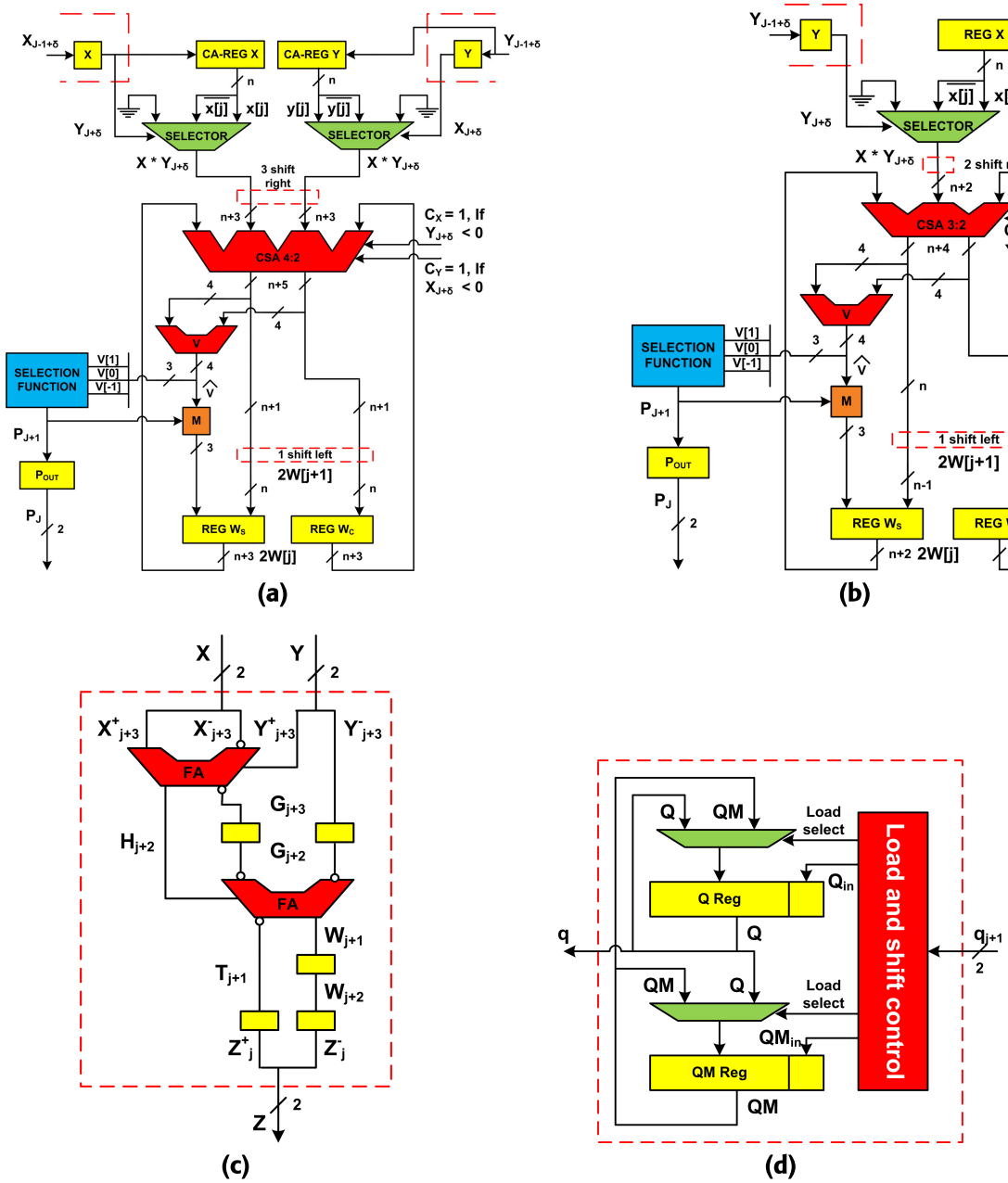

FIGURE 2. Different online arithmetic units. (a) OSSM [14]. (b) OSPM [15]. (c) OA [14]. (d) OFC [21].

TABLE 1. Radix-2 multiplication selection function.

$\widehat{V}$	$v_{-1}v_0.v_1$	P_{j+1}
3/2	01.1	1
1	01.0	1
1/2	00.1	1
0	00.0	0
-1/2	11.1	0
-1	11.0	-1
-3/2	10.1	-1
-2	10.0	-1

that the estimate bit v_2 is not used, the SELM is described by Table 1. The M module subtracts the $\widehat{V}$ from P_{j+1}. It can simply does that by complementing the estimate bit v_0 if $P_{j+1} \neq 0$. The output of the M module along with the output of CSA 4:2 form the new residuals according to Algorithm 1 to be stored in REG W_S and REG W_C. P_j is the output digit of OSSM.

$$P_{j+1} = SELM(\widehat{V}[j]) = \begin{cases} 1 & if\ \frac{1}{2} \leq \widehat{V}[j] \leq \frac{7}{4} \\ 0 & if\ -\frac{1}{2} \leq \widehat{V}[j] \leq \frac{1}{4} \\ -1 & if\ -2 \leq \widehat{V}[j] \leq -\frac{3}{4} \end{cases} \quad (1)$$

2) ONLINE SERIAL-PARALLEL MULTIPLIER (OSPM)

Radix-2 OSPM is depicted in Fig. 2 (b) [15]. It takes one input (X) in parallel manner and second (Y) in serial manner. δ for OSPM (δ_M) is 2. It follows the process of Algorithm 2 [15]. For n-digit input, it consists of $n + \delta_M$ iterations. One digit of serial input (Y) is introduced during cycles -δ_M to $n - 1 - \delta_M$. After that input is kept 0 for rest of cycles. Initially the output (P_{out}) is 0 for cycles -δ_M to -1. After that one digit of output (P_{out}) is produced in each cycle starting from cycle 0. The derivation of the algorithm for OSPM is presented in [22]. Like OSSM, OSPM also processes the partial products internally in carry-save form (W_s and W_c) and produces the output (P_{out}) in signed-digit form in the range of $(-1, 1)$.

As seen in Fig. 2 (b), OSPM works such that the input X comes in parallel form and Y comes in MSDF form. The REG stores the parallel input X. Then, the SELECTOR chooses the partial product to be added based on the input digit Y. If the input digit is 0 then the SELECTOR outputs 0, 1 then the SELECTOR outputs the output of REG and -1 then the SELECTOR outputs the inverted output of REG. The CSA 3:2 adds the partial product with the residual stored in REG W_S and REG W_C. Here the input carries C_X corresponds to the sign of the MSDF input digit Y. The V module consist of a 4-bit CPA. It produces the estimate $\widehat{V}$ i.e. $(v_{-1}v_0v_1v_2)$ by adding the first 4-bit of the carry save format output of CSA 3:2. SELECTION FUNCTION (SELM) is the product-digit selection function which selects the output digit based on Eq. 1 and Table 1 same as OSSM. The M module subtracts the $\widehat{V}$ from P_{j+1}. It can simply does that by complementing the estimate bit v_0 if $P_{j+1} \neq 0$. The output of the M module along with the output of CSA 3:2 form the new residuals according to Algorithm 2 to be stored in REG W_S and REG W_C. P_j is the output digit of OSPM.

Algorithm 2 Radix-2 OSPM Algorithm

procedure $OSPM(X, Y, P_{out})$
X: Parallel input (Element of input feature map)
Y: Serial input (Element of filter)
P_{out}: Serial output
1. [Initialize]
  $y[-2] = w[-2] = 0$
  for$(j = -2, -1)$ **do**
    $v[j] = 2w[j] + (X.y_{j+2})2^{-2}$
    $w[j+1] \leftarrow v[j]$
  endfor
2. [Recurrence]
  for$(j = 0, \ldots, n-1)$ **do**
    $v[j] = 2w[j] + (X.y_{j+2})2^{-2}$
    $p_{j+1} = SELM(\widehat{V}[j])$
    $w[j+1] \leftarrow v[j] - p_{j+1}$
    $P_{out} \leftarrow p_{j+1}$
  end for
end procedure

3) ONLINE ADDER (OA)

Radix-2 OA is depicted in Fig. 2 (c) [14]. It takes both inputs (X and Y) in serial manner. Both inputs are in signed-digit form in the range of $(-1, 1)$. Thus, it can take as input the output (P_{out}) of OSSM/OSPM. It also generates the output (Z) in signed-digit form. δ for OA (δ_A) is 2. It follows the computation of redundant adders such as signed-digit adder or CSA. The serialization of such redundant adders result in an OA. It involves two step addition, in a way that intermediate terms are so generated that the final addition requires no carry propagation.

TABLE 2. Parameters of a convolution layer.

Parameter	Description
H_{in}/W_{in}	Height/Width of IFM
H_{out}/W_{out}	Height/Width of OFM
H_f/W_f	Height/Width of filter
C_{in}/C_{out}	No. of channels of IFM/OFM

4) ON THE FLY CONVERTER (OFC)

In online arithmetic based computing it is required to convert the output signed-digit number into conventional BNS representation. This conversion requires an addition after the complete generation of signed-digit output. It requires CPA. However, this additional step increases the time to get the final output. To avoid this additional time for conversion, one can use OFC [21]. Block diagram of OFC is shown in Fig. 2 (d). It performs the conversion of signed-digit number coming in MSDF form to conventional number in a digit-serial manner. According to Eq. 2 the conversion concatenates the incoming serial digit (q_{j+1}) until $q_{j+1} \geq 0$. When $q_{j+1} < 0$, we have to perform subtraction which needs carry propagation. To avoid carry propagation OFC also keeps the output in another form QM. Thus, when $q_{j+1} < 0$ it loads from QM instead of Q and concatenates. It updates QM according to Eq. 3. It produces the conventional output in BNS (q) along with generation of signed-digit output from online arithmetic units [14]. Thus, OFC does not require additional time. In terms of concatenation for radix-2 the algorithm for OFC is given as

$$Q[j+1] = \begin{cases} (Q[j], q_{j+1}) & if\ q_{j+1} \geq 0 \\ (QM[j], (2 - |q_{j+1}|)) & if\ q_{j+1} < 0 \end{cases} \quad (2)$$

$$QM[j+1] = \begin{cases} (Q[j], q_{j+1} - 1) & if\ q_{j+1} > 0 \\ (QM[j], (1 - |q_{j+1}|)) & if\ q_{j+1} \leq 0 \end{cases} \quad (3)$$

With the initial condition Q[0] = QM[0] = 0 (for positive quotient) [14].

IV. ON-CNN: ONLINE ARITHMETIC BASED CNN ACCELERATOR

A. BASELINE DATA-FLOW FOR CONVOLUTION COMPUTATION

The first research challenge that we have identified in section II is to process filter of any dimension. To address this

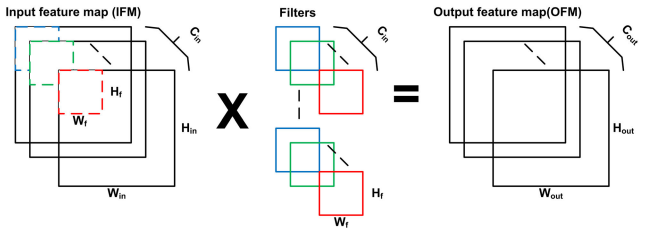

FIGURE 3. Computation of a convolution layer.

challenge we have employed an efficient data-flow [12], [13]. The primary computation kernel in the convolution layer is performing the high-dimensional convolutions, summarized in Fig. 3. Table 2 shows the parameters of a convolution layer. In a layer of CNN, we have an IFM, which consists of multiple channels. We also have multiple number of filters. Each filter has number of channels making it a 3D object. Collectively we call multiple 3D filters a 4D object. A layer applies multiple 3D filters i.e. 4D filter with a 3D IFM and generates multiple 2D OFM, which combines to form a 3D OFM.

Let us consider 2D convolution first to understand the baseline data-flow. Fig. 4 shows computation of a 2D convolution layer. The IFM divides into total H_{out} x W_{out} IFM windows. Each with dimension similar to a filter size i.e. H_f x W_f. In order to perform 2D convolution, we propose to employ a row of PEs. In a row, we propose to employ C number of PEs. We choose C to be 16. Each PE processes one IFM window. Thus, a row can process 1 filter and 16 IFM windows at one time. Each PE processes one element of IFM window and filter at a time. It multiples an element of IFM window with corresponding element in filter and stores the result in an accumulator register. In the next cycle it multiplies the next element of IFM window and filter. And adds the result to the result of previous computation. Thus, a PE takes $\lceil H_f \times W_f \rceil$ cycles to compute SOPs of all elements of IFM window and filter. If the number of IFM windows are greater than C, then it uses a row of PEs multiple times to processes all IFM windows. Thus, it has to use a row $\lceil H_{out} \times W_{out}/C \rceil$ times. The total number of computational cycles (Comp) are given as

$$Comp = \left\lceil \frac{H_{out} \times W_{out}}{C} \right\rceil \times \lceil H_f \times W_f \rceil \tag{4}$$

Now we will consider 3D convolution. As IFM and filter have multiple channels (C_{in}) which make them 3D. Thus, a PE takes $\lceil C_{in} \times H_f \times W_f \rceil$ cycles to compute SOPs of all elements of all channels of 3D IFM window and filter. Fig. 5 shows computation of a 3D convolution layer. Instead of employing 1 multiplier in a PE, we propose to employ M number of multipliers in a PE followed by an adder tree. We choose M to be 16. Each multiplier of a PE processes one channel. Thus, a PE can process M channels at a time. If we have more than M channels, then we use a PE multiple times to processes all C_{in} channels. Thus, a PE will be

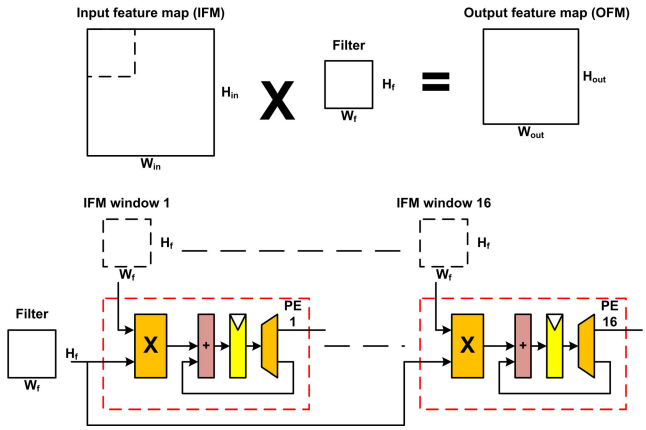

FIGURE 4. Computation of a 2D convolution layer.

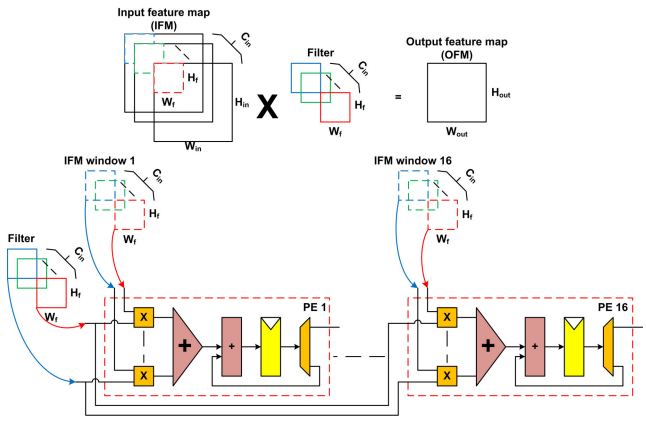

FIGURE 5. Computation of a 3D convolution layer.

used $\lceil (C_{in} \times H_f \times W_f)/M \rceil$ times. Now the total number of computational cycles are given as

$$Comp = \left\lceil \frac{H_{out} \times W_{out}}{C} \right\rceil \times \left\lceil \frac{C_{in} \times H_f \times W_f}{M} \right\rceil \tag{5}$$

Till now, we have considered a 3D filter. But in CNN there are multiple 3D filters i.e. a 4D filter. To process C_{out} number of filters, we propose to employ multiple rows of PEs. Each row processes a filter. We propose to employ R number of rows. We choose R to be 16. Now the design becomes a tile i.e. 2D mesh with RxC dimension. Thus, for C_{out} filters, we have to use a tile $\lceil C_{out}/R \rceil$ times. But in modern CNNs there are huge number of filters. Thus, to reduce the number of iterations of a tile, we propose to employ T number of tiles. We choose T to be 16. Now to process C_{out} filters, 16 tiles will be used $\lceil C_{out}/(T \times R) \rceil$ times. The total number of computational cycles are given as

$$Comp = \left\lceil \frac{C_{out}}{T \times R} \right\rceil \times \left\lceil \frac{H_{out} \times W_{out}}{C} \right\rceil \times \left\lceil \frac{C_{in} \times H_f \times W_f}{M} \right\rceil \tag{6}$$

The proposed CNN accelerator architecture (ON-CNN) consists of 16 tiles. Each tile consists of 16 rows and

16 columns, which make up 2D mesh of PEs. Each PE consists of 16 multipliers followed by adder tree. Eq. 6 consists of 3 terms. Each term evaluates the number of executions. The first term evaluates the number of times the accelerator needs to run to execute all filters. The second term evaluates the number of times each tile needs to run in each iteration of accelerator to execute all IFM windows. The third term evaluates the number of times a PE needs to run in each iteration of a tile.

Algorithm 3 shows the associated procedure of convolution for the baseline data-flow. It can be seen that the employed data-flow can process filters of any dimension. Previous data-flows of state-of-the-art CNN accelerator architectures have chosen number of PEs depending upon dimension of filter [7], [8], [10], [11]. The most common choice was 3 PEs which only works well for filter of dimension 3×3. As 3 is not a multiple of other common filter dimensions like $25(5 \times 5)$, $49(7 \times 7)$, $121(11 \times 11)$ etc. Thus, these data-flows are unable to process these filter dimensions or suffer from under-utilization while processing. For the employed data-flow, the choice of multipliers inside a PE, number of rows, columns and tiles do not depend upon dimension of a filter. A PE comprises of 16 multipliers followed by adder tree. Each multiplier of a PE processes one channel of filter/IFM window. Thus, a PE can processes 16 channels of filter/IFM window at a time. A multiplier processes one element of channel of filter/IFM window at a time. Thus, a PE takes H_f x W_f cycles to process a channel of filter/IFM window. For example, for convolution of a 3×3 filter and IFM window, a multiplier takes 9 cycles, for 5×5 filter 25 cycles, for 7×7 filter 49 cycles and for 11×11 filter 121 cycles. Thus, the employed data-flow can process filter of any dimension without under-utilization of PE resources as long as number of channels are multiple of 16, which is mostly the case.

Algorithm 3 Convolution Layer Execution

procedure $CONVOLUTION(H_{out}, W_{out}, C_{out}, C_{in}, H_f, W_f)$
H_{out} x W_{out}: number of IFM windows
C_{out}: number of input filters
C_{in}: number of channels
H_f x W_f: dimension of filter
IN: input
OUT: output
W: filter
for$(w = 0; w < (H_{out} \text{ x } W_{out}); w += C)$ **do**
  for$(f = 0; f < C_{out}; f += R)$ **do**
    for$(c = 0; c < (C_{in} \text{ x } H_f \text{ x } W_f); c += M)$ **do**
      return $OUT[f][H_{out}][W_{out}] += IN[c][H_{in}][W_{in}]$ x $W[f][c][H_f][W_f]$
    end for
  end for
end for
end procedure

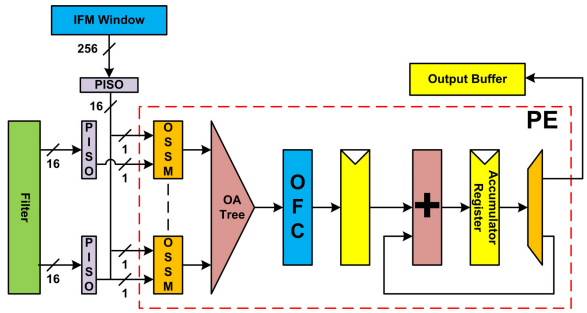

FIGURE 6. ON-CNN$_{SS}$ PE architecture.

B. PROPOSED ONLINE ARITHMETIC BASED EXECUTION APPROACH

The second research challenge that we have identified is to reduce the execution time for computation of a convolution layer. In order to address this challenge, we propose online arithmetic based execution approach. Up till now, we have considered that a PE computes 16 SOPs in a cycle and employ one register as an accumulator register to compute more than 16 SOPs. But we are employing online arithmetic units which are serial and iterative in nature. In cascade they form an inherent pipeline which takes number of cycles to compute 16 SOPs. Thus, a different PE architecture is required to support proposed online arithmetic based execution approach. The proposed PE comprises of 16 OSSM/OSPM followed by 4 levels of OA tree and an OFC. It has two versions: ON-CNN$_{SS}$ which employs OSSM and ON-CNN$_{SP}$ which employs OSPM. ON-CNN$_{SS}$ takes both inputs in serial form as shown in Fig. 6. Whereas, ON-CNN$_{SP}$ takes one input in parallel and second in serial form as shown in Fig. 7. Both PEs employ parallel-in-serial-out (PISO) shift registers at inputs to provide the serial inputs. Parallel-to-serial conversion does not affect the execution time. On the contrary, it improves the execution time by enabling online processing. OFC generates the output in parallel form. Thus, the input and output interfaces of a PE process numbers in parallel form.

Both ON-CNN$_{SS}$ and ON-CNN$_{SP}$ PEs take elements of IFM window in serial form. Whereas, ON-CNN$_{SS}$ PE takes elements of filter in serial form and ON-CNN$_{SP}$ takes elements of filter in parallel form. The PE computes 16 SOPs. As soon as an online arithmetic unit produces MSD, the next unit starts computation. The timing diagrams of the proposed online execution approach for 2 consecutive computations using ON-CNN$_{SS}$ PE is shown in Fig. 8(a) and using ON-CNN$_{SP}$ PEs is shown in Fig. 8(b). After δ_M cycles the OSSM/OSPM produces the first output digit starting from MSD. OA tree level 1 adders take the MSD outputs from OSSM/OSPM as input and generates the MSD output after δ_A cycles. Each subsequent OA takes δ_A cycles to produce the first MSD output. OFC follows the level 4 of OA tree. It converts the signed-digit output to conventional BNS representation. After each 16 SOPs computation one

cycle is required to reset (R_C) the internal registers of OSSM/OSPM, OA and OFC. S_C represents the separation cycles between application/generation of input/output. It can be seen that ON-CNN$_{SS}$ PE takes 47 clock cycles to compute 16 SOPs. Whereas, ON-CNN$_{SP}$ PE takes 46 clock cycles to compute 16 SOPs. The 1 clock cycle difference is because δ_M for OSSM is 3 and δ_M for OSPM is 2.

Inside each PE a register follows OFC. As it takes number of cycles to mature the result of 16 SOPs. This register stores the 16 SOPs result. Each PE also has an accumulator register to store the accumulated result of all previous iteration. It needs an accumulator register in case, one needs to compute more SOPs than the number of multipliers inside a PE. For accumulation it employs CPA. Due to carry propagation delay, it takes more time for accumulation. Thus, the use of two registers enables the accumulation along with computation for the current iteration.

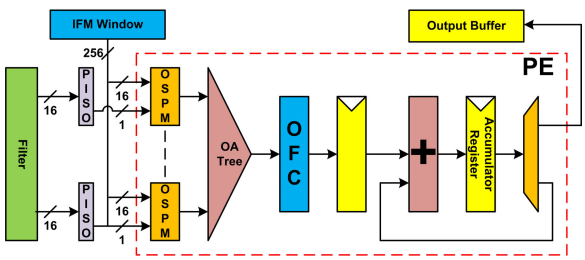

FIGURE 7. ON-CNN$_{SP}$ PE architecture.

The use of online arithmetic units do not affect the employed data-flow. They only change the third term of Eq. 6. As it deals with iterations of a PE. To compute 16 SOPs, ON-CNN PE takes following number of cycles

$$C_{SOP} = \delta_M + \delta_A \times \lceil \log_2(M) \rceil + 2 \times P_a + \lceil \log_2(M) \rceil \quad (7)$$

where P_a represents the input bit precision of activation i.e. IFM window. For the proposed architecture P_a is 16-bit. The term $\lceil \log_2(M) \rceil$ represents the number of OA tree levels. In the last term it represents the total bit-growth of the OA tree. Now each run of a PE consists of $C_{SOP} + 1$ cycles. Here, we have added 1 to C_{SOP} to count a reset cycle too. If one needs to compute more than 16 SOPs then we need to compute the PE more than once in each iteration of a tile i.e. N times. Then for the proposed online execution approach the number of times a PE needs to run in each iteration of a tile is given as

$$N_{ON-CNN} = (C_{SOP} + 1) \times N \quad (8)$$

For the employed data-flow N represents the third term in Eq. 6. Thus, the modified third term N_{ON-CNN} for the proposed online execution approach based data-flow is given as

$$N_{ON-CNN} = \left\lceil (C_{SOP} + 1) \times \frac{C_{in} \times H_f \times W_f}{M} \right\rceil \quad (9)$$

For ON-CNN accelerator architecture, the total number of computation cycles ($Comp_{ON-CNN}$) required for computation of a convolution layer based on Eq. 6 and 9 are given as

$$Comp_{ON-CNN} = \left\lceil \frac{C_{out}}{T \times R} \right\rceil \times \left\lceil \frac{H_{out} \times W_{out}}{C} \right\rceil \times \left\lceil (C_{SOP} + 1) \times \frac{C_{in} \times H_f \times W_f}{M} \right\rceil \quad (10)$$

Example: Let's calculate the computation cycles required by ON-CNN accelerator architecture to compute CONV2-2 layer of ResNet-18 [23] which has $C_{out} = 64$, H_{out} x $W_{out} = 56 \times 56$, $C_{in} = 64$ and H_f x $W_f = 3 \times 3$.

For ON-CNN$_{SS}$ accelerator architecture, Eq. 10 predicts following number of computation cycles

$$Comp_{ON-CNN_{SS}} = \left\lceil \frac{64}{16 \times 16} \right\rceil \times \left\lceil \frac{56 \times 56}{16} \right\rceil \times \left\lceil (47 + 1) \times \frac{64 \times 3 \times 3}{16} \right\rceil = 338688$$

For ON-CNN$_{SP}$ accelerator architecture, Eq. 10 predicts following number of computation cycles

$$Comp_{ON-CNN_{SP}} = \left\lceil \frac{64}{16 \times 16} \right\rceil \times \left\lceil \frac{56 \times 56}{16} \right\rceil \times \left\lceil (46 + 1) \times \frac{64 \times 3 \times 3}{16} \right\rceil = 331632$$

ON-CNN$_{SS}$ accelerator architecture takes more computational cycles in comparison with ON-CNN$_{SP}$ accelerator architecture because of difference in value of δ_M.

C. PROPOSED ON-CNN ACCELERATOR ARCHITECTURE

The architecture of the proposed online arithmetic based accelerator (ON-CNN) is shown in Fig. 9. It can perform matrix-matrix multiplication i.e. convolution computation and matrix-vector multiplication i.e. FC layer computation. At the heart, it has a computational core. The computational core consists of 16 tiles. In each tile 16 × 16 PEs are arranged in 2D grid. This 16 × 16x16 array accelerator configuration has been widely used in research to propose different computing and re-configuration techniques. DaDianNao [24] employs one column per tile configuration (16 × 1x16). Stripes [12] and Loom [13] have used 16 × 16x16 configuration to propose serial-parallel and serial-serial bit-serial computing. Bit-pragmatic [25] and Laconic [26] have used 16 × 16x16 configuration to propose sparsity aware bit-serial computing. RASHT [27] and REI [28] have used 16 × 16x16 configuration to propose re-configuration approaches to reduce the number of computational cycles. Thus, to show the significance of the proposed online arithmetic based execution approach for CNN acceleration and proposed re-configurable power saving strategy, we have chosen the same popular configuration.

ON-CNN consists of two on-chip memories: input memory and output memory. Input memory stores the IFM windows and filters. Whereas output memory stores the computed OFM. Both of the on-chip memories have interface with the off-chip memory. It also has a separate post-processing unit (PPU) to perform ReLU activation and max pooling. PPU performs these computations before storing the data in output

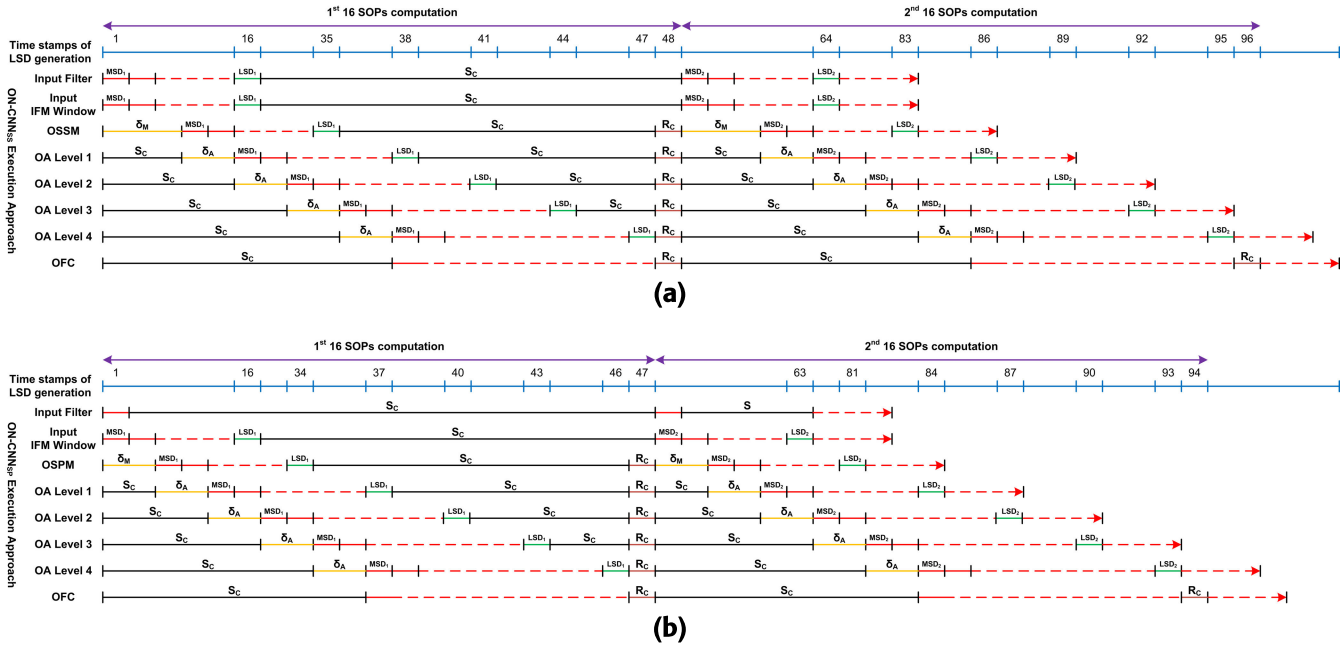

FIGURE 8. Timing diagram of proposed online execution approaches. S_C represents separation cycles and R_C represents reset cycle. (a) ON-CNN$_{SS}$ execution approach. (b) ON-CNN$_{SP}$ execution approach.

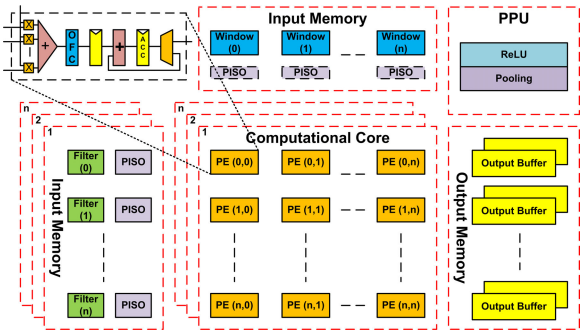

FIGURE 9. Proposed ON-CNN accelerator architecture (n = 16).

buffers. These computations do not require improvement as they have negligible contribution towards the latency.

In ON-CNN, the operation of each PE does not depend on the output of neighboring PEs. Each column takes the same IFM window and each row takes the same filter. Thus, all PEs work in parallel to each other. The control unit performs following steps for performing a convolution computation:

1) It fetches the IFM windows and filters from off-chip memory and loads them to input memory.
2) It feeds the IFM windows across columns and filters across rows.
3) The PE produces the SOP result.
4) The accumulator register accumulates the SOP results until the computation for a layer has finished.
5) After computation for a layer, it propagates the output data to the output memory and uses it as input for the next layer.

ON-CNN can also compute FC layers. FC layers consist of large number of filters with much larger dimensions in comparison with convolution layers. In FC layers, an input window multiplies with many filters. Since computation of FC layers can not take advantage of filter reuse, they need only one column of each tile. However, the focus of this work is only for the acceleration of convolution layers. Thus, we have kept our focus towards convolution layers. The total execution time (T_{ON-CNN}) for computation of a convolution layer is given as

$$T_{ON-CNN} = Comp_{ON-CNN} \times C_{ON-CNN} \tag{11}$$

where C_{ON-CNN} is the clock period for ON-CNN accelerator. The synthesis result with NanGate 45nm FreePDK technology library shows that the clock period for the ON-CNN$_{SS}$ architecture is 1.2 ns and ON-CNN$_{SP}$ architecture is 1.1 ns. Thus, the total execution time for CONV2-2 layer of ResNet-18 for ON-CNN$_{SS}$ is $338688 \times 1.2\ ns = 0.406\ ms$ and ON-CNN$_{SP}$ is $331632 \times 1.1\ ns = 0.365\ ms$.

D. PROPOSED RE-CONFIGURABLE POWER SAVING STRATEGY

The third research challenge that we have identified in section II is to design a power-efficient architecture. In order to address this challenge, we propose a re-configurable power saving strategy. ON-CNN comprises of 4096 PEs. Not all layers require all the PEs for computation. Some of the PEs are idle in computation of different layers. Table 3 shows the percentage of idle rows, columns and multipliers in execution of each convolution layer for VGG-16. It can be seen that some resources are idle. They consume a lot of

TABLE 3. Percentage of idle resources while executing different convolution layers of VGG-16.

Layer No.	Layer Characteristics				Architecture Characteristics				Percentage Idle Resources		
	C_{out}	H_{out}/W_{out}	C_{in}	H_f/W_f	T	R	C	M	R	C	M
1	64	224	3	3	16	16	16	16	75	0	81.25
2	64	224	64	3	16	16	16	16	75	0	0
3	128	112	64	3	16	16	16	16	50	0	0
4	128	112	128	3	16	16	16	16	50	0	0
5	256	56	128	3	16	16	16	16	0	0	0
6	256	56	256	3	16	16	16	16	0	0	0
7	256	56	256	3	16	16	16	16	0	0	0
8	512	28	256	3	16	16	16	16	0	0	0
9	512	28	512	3	16	16	16	16	0	0	0
10	512	28	512	3	16	16	16	16	0	0	0
11	512	14	512	3	16	16	16	16	0	5.769	0
12	512	14	512	3	16	16	16	16	0	5.769	0
13	512	14	512	3	16	16	16	16	0	5.769	0

TABLE 4. Computational cycles comparison without tile gating and with tile gating mechanism for VGG-16.

Layer No.	Layer Characteristics				Architecture Characteristics			Computation Cycles Without Tile Gating			Computation Cycles With Tile Gating		
	C_{out}	H_{out}/W_{out}	C_{in}	H_f/W_f	R	C	L	T	ON-CNN$_{SS}$	ON-CNN$_{SP}$	T	ON-CNN$_{SS}$	ON-CNN$_{SP}$
1	64	224	3	3	16	16	16	16	254016	250880	4	254016	250880
2	64	224	64	3	16	16	16	16	5419008	5306112	4	5419008	5306112
3	128	112	64	3	16	16	16	16	1354752	1326528	8	1354752	1326528
4	128	112	128	3	16	16	16	16	2709504	2653056	8	2709504	2653056
5	256	56	128	3	16	16	16	16	677376	663264	16	677376	663264
6	256	56	256	3	16	16	16	16	1354752	1326528	16	1354752	1326528
7	256	56	256	3	16	16	16	16	1354752	1326528	16	1354752	1326528
8	512	28	256	3	16	16	16	16	677376	663264	16	677376	663264
9	512	28	512	3	16	16	16	16	1354752	1326528	16	1354752	1326528
10	512	28	512	3	16	16	16	16	1354752	1326528	16	1354752	1326528
11	512	14	512	3	16	16	16	16	359424	351936	16	359424	351936
12	512	14	512	3	16	16	16	16	359424	351936	16	359424	351936
13	512	14	512	3	16	16	16	16	359424	351936	16	359424	351936

power. It makes ON-CNN architecture less power-efficient. In order to deal with low power-efficiency, we propose to switch-off idle PEs depending upon characteristics of each convolution layer. By carefully observing the characteristics of ON-CNN architecture and convolution layers, we propose to use suitable number of tiles and columns for computation of each layer. We propose two gating mechanisms: tile gating and column gating. They do not affect the number of computation cycles and save significant percentage of power.

Table 4 shows the computation cycles comparison for all convolution layers of VGG-16 for ON-CNN$_{SS}$ and ON-CNN$_{SP}$ without tile gating and with tile gating mechanism. The first term of Eq. 10 is of importance here. The numerator is the characteristics of a convolution layer. Whereas, the denominator is characteristics of ON-CNN architecture. In the first term of Eq. 10, C_{out} in the numerator shows the total number of filters in a layer. Whereas, TxR in the denominator represents the number of filters ON-CNN can process. Now consider layers no. 1-2 of VGG-16. They contain 64 filters. ON-CNN needs only 4 tiles i.e. 4×16 to process 64 filters. The remaining 12 tiles are idle and can be switched-off using tile gating mechanism. Thus, roughly 75% power can be saved for layers no. 1-2 of VGG-16. Layers no. 3-4 have 128 filters and need only 8 tiles i.e. 8×16 for processing. The remaining 8 tiles are idle and can be switched-off. Thus, roughly 50% power can be saved for layers no. 3-4 of VGG-16. Layers no. 5-7 need all 256 tiles. Whereas, layers no. 8-13 need 512 tiles. Thus, for layers no. 8-13 ON-CNN operates two times. It can be seen that computation cycles for without tile gating and with tile gating execution is same. Tile gating just saves the power by switching-off some tiles according to the requirement by a layer.

The second term of Eq. 10 also is of importance. H_{out} x W_{out} in the numerator shows the total number of IFM windows. Whereas, C in the denominator represents the number of IFM windows ON-CNN can process. Now consider layers no. 11-13 of VGG-16. They contain 196 IFM windows. They need 12.25 iterations according to the second term. After applying ceiling function, it becomes 13 iterations. It means that in the 13th iteration there are idle columns. In each iteration for the second term, ON-CNN can process 16 IFM windows. In the first 12 iterations ON-CNN processes 192 IFM windows. In the 13th iteration, ON-CNN processes remaining 4 IFM windows using the first 4 columns. Thus, PEs in the remaining 12 columns are

idle. We propose to use column gating to switch-off the idle columns in the 13th iteration to save power. Thus, roughly 5.769% power can be saved for layers no. 11-13 of VGG-16.

The third term of Eq. 10 is of little importance for power saving. C_{in} in the numerator shows the total number of input channels. Whereas, M in the denominator represents the number of input channels a PE can process. One can exploit it for multiplier gating when number of input channels (C_{in}) are not the integer multiple of number of multipliers (M) in a PE. For example, in the case of layer no. 1 of VGG-16. It contains 3 input channels. Thus, it require only 3 multipliers to process. The remaining multipliers will take zero input and are idle. Multiplier gating requires fine grain gating mechanism at the level of multipliers and adders. It also requires bypass paths. Whereas, in CNN models the number of input channels increase after first layer. Thus, it is not beneficial to employ multiplier gating. Hence, we are not employing multiplier gating in the proposed architecture. For both ON-CNN$_{SS}$ and ON-CNN$_{SP}$ architectures, the proposed power saving strategy saves overall 19.2% power while executing VGG-16, 33.8% power while executing ResNet-18 and 18.9% power while executing ResNet-50. Thus, in spite of employing large number of PEs we are able to save significant amount of power.

V. RESULT AND ANALYSIS

We have implemented ON-CNN$_{SS}$ and ON-CNN$_{SP}$ accelerators in Verilog. For verification we have used Synopsys VCS. For synthesis, we used NanGate 45*nm* FreePDK technology library with Synopsys Design Compiler. For power consumption measurement, we have used PrimePower.

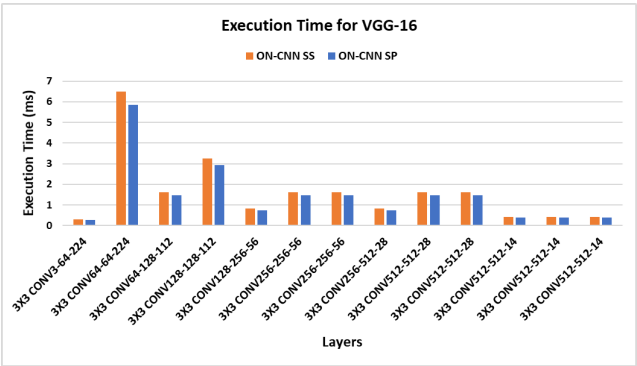

FIGURE 10. Execution time comparison for VGG-16 using ON-CNN$_{SS}$ and ON-CNN$_{SP}$ architectures.

A. EXECUTION TIME AND THROUGHPUT ANALYSIS

We have computed the execution time for convolution layers of popular CNN models like VGG-16 [4], ResNet-18 and ResNet-50 [23]. All these CNN models have won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) [29]. VGG-16 consists of 13 convolution layers having 14.7M weights. It performs 30.6G operations in convolution layers. ResNet exploits residual learning for training and has low-complexity. ResNet-18 consists of 17 convolution layers having 11M weights. It performs 3.6G operations in convolution layers. Whereas, ResNet-50 consists of 49 convolution layers having 23.5M weights. It performs 7G operations in convolution layers.

Fig. 10, 11 and 12 shows the execution time comparison of ON-CNN$_{SS}$ and ON-CNN$_{SP}$ for VGG-16, ResNet-18 and ResNet-50 respectively. It can be seen that ON-CNN$_{SP}$ exhibits less execution time in comparison with ON-CNN$_{SS}$. It is because δ_M for OSPM of ON-CNN$_{SP}$ is 2, whereas δ_M for OSSM of ON-CNN$_{SS}$ is 3. The difference in the value of δ_M results in difference in number of computation cycles. Thus, execution time of ON-CNN$_{SS}$ is more in comparison with ON-CNN$_{SP}$.

Table 5 shows the execution time comparison for state-of-the-art architectures with ON-CNN$_{SS}$ and ON-CNN$_{SP}$ for VGG-16, ResNet-18 and ResNet-50. Most of the architectures report results using 65 nm technology library. Whereas we have used 45 nm technology library. For fair comparison, we have scaled the execution time for proposed ON-CNN$_{SS}$ and ON-CNN$_{SP}$ architectures from 45 nm to 65 nm technology using the scaling method of [30]. The scaled frequencies for ON-CNN$_{SS}$ and ON-CNN$_{SP}$ are 593 and 648 MHz respectively.

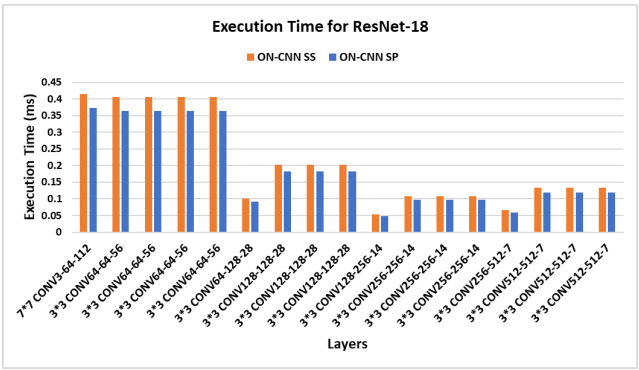

FIGURE 11. Execution time comparison for ResNet-18 using ON-CNN$_{SS}$ and ON-CNN$_{SP}$ architectures.

Execution time depends upon different characteristics of the architecture: data-flow, size of architecture, arithmetic units etc. Data-flow plays an important role in determining the computational ability of an architecture. It can be seen in Table 5 that Eyeriss takes the most execution time for computing convolution layers of VGG-16. Row-stationary data-flow of Eyeriss focuses on reducing the number of memory accesses. And does not contribute toward reducing the execution time. FID, ZASCA, CARLA and BLMAC have same data-flow. They focus on computation of CNN workload using FC-layer methods. FID specially focuses on improving the execution time for VGG-based networks. ZASCAD and ZASCAS can process any CNN model. Further, ZASCAS exploits sparsity in activation to reduce the execution time. On the other hand, CARLA focuses on reducing the under-utilization of PEs to achieve higher

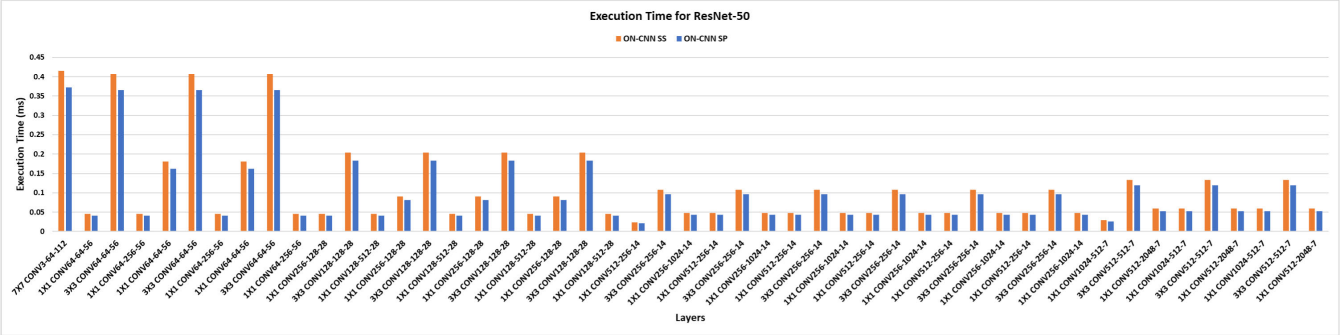

FIGURE 12. Execution time comparison for ResNet-50 using ON-CNN$_{SS}$ and ON-CNN$_{SP}$ architectures.

TABLE 5. Execution time comparison of the proposed ON-CNN$_{SS}$ and ON-CNN$_{SP}$ CNN accelerator architectures with state-of-the-art CNN accelerator architectures for VGG-16, ResNet-18 and ResNet-50. Note that the execution time of the architectures is taken from their original papers.

Architecture	Year	Technology (nm)	Frequency (MHz)	No. of PEs	Execution time (ms)		
					VGG-16	ResNet-18	ResNet-50
Eyeriss [6]	2017	65	250	168	4309.5	-	-
Stripes [12]	2017	45	154	4096	38.1	6.5	9.8
Loom [13]	2018	45	182	4096	515.9	87.8	132.2
Stripes [12]	2017	65 (scaled)	100	4096	58.4	9.9	15.0
Loom [13]	2018	65 (scaled)	119	4096	790.5	134.6	202.5
FID (A-E) [7]	2018	65	400	96	436.4	-	-
FID (MA-E) [7]	2018	65	200	192	453.3	-	-
ZASCAD [8]	2020	65	200	192	421.8	56	103.6
ZASCAS [8]	2020	65	200	192	246.8	30.4	59.9
CARLA [10]	2021	65	200	196	396.9	-	92.7
BLMAC [11]	2022	65	962	92	169.1	-	-
ON-CNN$_{SS}$	2024	45	834	4096	21.1	3.6	5.4
ON-CNN$_{SP}$	2024	45	909	4096	19.0	3.2	4.9
ON-CNN$_{SS}$	2024	65 (scaled)	593	4096	29.6	5.0	7.6
ON-CNN$_{SP}$	2024	65 (scaled)	648	4096	26.6	4.5	6.8

utilization of resources. It achieves least execution time in comparison with Eyeriss, FID and ZASCA. ON-CNN$_{SS}$ and ON-CNN$_{SP}$ employ the same data-flow as of Stripes and Loom which processes IFM windows parallel with multiple filters and focuses on reducing the computational cycles. Thus, they achieve least execution time in comparison with state-of-the-art CNN accelerators.

Size of architecture determines the computational capacity of the architecture. Table 5 shows the number of PEs for each architecture. It can be seen that ON-CNN$_{SS}$ and ON-CNN$_{SP}$ employ the highest number of PEs. Eyeriss, FID, ZASCA, CARLA and BLMAC use conventional arithmetic units that process data in parallel form. Bit-parallel arithmetic units process more number of bits in parallel in comparison to conventional bit-serial and online arithmetic units. Even with small number of PEs they can compute more number of SOPs in certain amount of time. Thus, there throughput is higher. Whereas, ON-CNN$_{SS}$ and ON-CNN$_{SP}$ exploit online arithmetic units which process data in online form. In order to match the throughput with bit-parallel accelerators, just like Stripes and Loom, ON-CNN$_{SS}$ and ON-CNN$_{SP}$ employ more number of PEs. Thus, they achieve least execution time in comparison with state-of-the-art CNN accelerators.

The arithmetic data-path of an architecture determines the achievable frequency. Execution time scales inversely with frequency. Table 5 shows the frequency of each architecture. Eyeriss uses conventional two-stage pipelined multiplier and adder. Whereas, FID, ZASCA and CARLA employ conventional non-pipelined arithmetic units. Pipelinning increases the frequency of operation. Thus, frequency of Eyeris is higher in comparison with FID, ZASCA and CARLA. BLMAC uses higher-radix multiplier which consists of modified booth encoder and Wallace tree. It also uses CSA to avoid carry propagation. This way, it shortens the critical path and increases the frequency. BLMAC operates at 962 MHz, which is even higher than the frequencies of the proposed ON-CNN$_{SS}$ and ON-CNN$_{SP}$ architectures. Stripes and Loom employ conventional bit-serial arithmetic units which use recurrence to perform partial product generation/reduction. There critical path consists of the partial product generation/reduction circuitry and accumulator adder. Thus, there frequencies are lower. ON-CNN$_{SS}$ and ON-CNN$_{SP}$ employ online arithmetic units. Online arithmetic units are iterative in nature and consist of registers. In cascade they form an inherent pipeline. Thus, the critical path becomes short and frequency of operation increases. Because of the higher frequency of ON-CNN$_{SS}$ and ON-CNN$_{SP}$, they achieve least

execution time for different CNN models. Execution time of ON-CNN$_{SP}$ is shorter in comparison with ON-CNN$_{SS}$. It is because the critical path of ON-CNN$_{SS}$ and ON-CNN$_{SP}$ PEs lie inside the OSSM and OSPM. The critical path consists of a SELECTOR (2-input MUX), a CSA, V module (4-bit CPA), SELECTION FUNCTION module (look-up table) and M module (an XOR gate). OSSM consists of 4:2 CSA. Whereas OSPM consists of 3:2 CSA. Thus, critical path of ON-CNN$_{SP}$ is shorter than ON-CNN$_{SS}$. Hence, frequency of ON-CNN$_{SP}$ is higher than ON-CNN$_{SS}$.

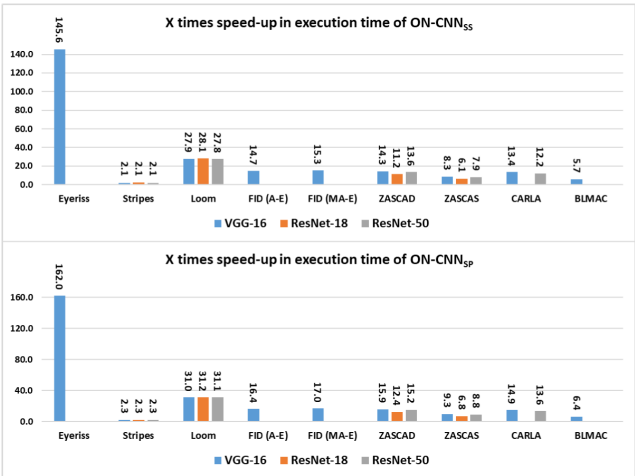

FIGURE 13. X times speed-up in execution time using ON-CNN$_{SS}$ and ON-CNN$_{SP}$ architectures.

Fig. 13 shows the speed-up in execution time of ON-CNN$_{SS}$ and ON-CNN$_{SP}$ in comparison with state-of-the-art CNN accelerators. ON-CNN$_{SS}$ shows speed-up of 145.6 times in comparison with Eyeriss to 5.7 times in comparison with BLMAC for VGG-16. Whereas, ON-CNN$_{SP}$ shows speed-up of 162.0 times in comparison with Eyeriss to 6.2 times in comparison with BLMAC for VGG-16. Eyeriss and ZASCAS also exploit sparsity in input activation to reduce the execution time. While, BLMAC employs higher-radix multiplier and CSA to shorten the critical path and improve the execution time. Stripes and Loom employ conventional bit-serial arithmetic units to improve the execution time. In comparison to these techniques, ON-CNN$_{SS}$ and ON-CNN$_{SP}$ only employ online arithmetic units. They achieve least execution time without exploiting sparsity. And they perform better than conventional higher-radix and bit-serial arithmetic based CNN accelerator architectures.

Given the parameters of a convolution layer and execution time (in ps), the throughput in terms of TOPS (Tera operations per second) is given as [31]

$$TOPS = \frac{2 \times C_{in} \times C_{out} \times H_{out} \times W_{out} \times H_f \times W_f}{Execution\ time} \quad (12)$$

Fig. 14 shows the peak throughput performance in terms of TOPS comparison of ON-CNN$_{SS}$ and ON-CNN$_{SP}$ with state-of-the-art CNN accelerators. It shows peak performance for AlexNet [32], VGG-16, ResNet-18 and ResNet-50. It can be seen that ON-CNN$_{SS}$ shows the highest performance of 2.276 TOPS for VGG-16, 2.144 TOPS for ResNet-18 and 2.276 TOPS for ResNet-50. Whereas, ON-CNN$_{SP}$ shows the highest performance of 2.535 TOPS for VGG-16, 2.389 TOPS for ResNet-18 and 2.535 TOPS for ResNet-50. ON-CNN$_{SP}$ shows higher throughput in comparison with ON-CNN$_{SS}$ because unlike ON-CNN$_{SS}$ it takes less execution time. In comparison with other state-of-the-art CNN accelerator architectures, both ON-CNN$_{SS}$ and ON-CNN$_{SP}$ show higher performance. This high performance is due to employing more number of PEs which makes ON-CNN$_{SS}$ and ON-CNN$_{SP}$ highly parallel architectures. One of the other reason is inherent pipeline of cascaded online arithmetic units, which results in higher achievable frequency.

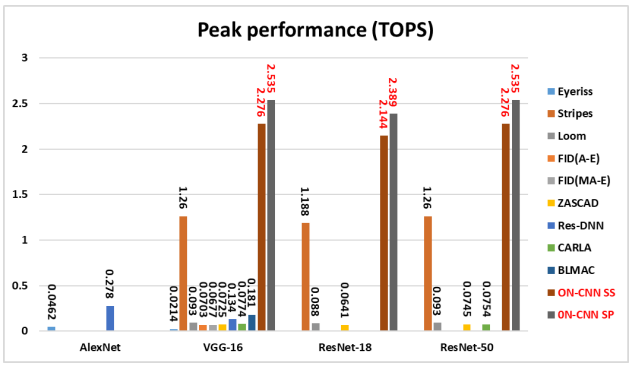

FIGURE 14. Peak performance comparison with state-of-the-art CNN accelerator architectures. Note that the peak performance of the architectures is taken from their original papers.

B. AREA AND POWER EFFICIENCY ANALYSIS

Table 6 presents the overall comparison of proposed ON-CNN$_{SS}$ and ON-CNN$_{SP}$ accelerator architectures with state-of-the-art CNN accelerator architectures. All the accelerator architectures support 16-bit input precision. For better visualization, Fig. 15 shows the area-efficiency comparison of proposed ON-CNN$_{SS}$ and ON-CNN$_{SP}$ accelerator architectures with state-of-the-art CNN accelerator architectures. It can be seen that area-efficiencies of proposed ON-CNN$_{SS}$ and ON-CNN$_{SP}$ accelerator architectures are better in comparison with most state-of-the-art CNN accelerator architectures. Among conventional bit-serial accelerator architectures, Stripes accelerator architecture shows better area efficiency in comparison with proposed ON-CNN$_{SS}$ and ON-CNN$_{SP}$ accelerator architectures. It shows high area-efficiency of 0.0693 TOPS/mm^2 for VGG-16. But does not exhibit better execution time and peak performance in comparison with proposed ON-CNN$_{SS}$ and ON-CNN$_{SP}$ accelerator architectures. Among bit-parallel accelerator architectures, BLMAC accelerator architecture shows highest area-efficiency of 0.0724 TOPS/mm^2 for VGG-16. Though BLMAC does not exhibit better execution time and peak performance in comparison with proposed ON-CNN$_{SS}$ and ON-CNN$_{SP}$ accelerator architectures. Res-

TABLE 6. Overall comparison of the proposed ON-CNN$_{SS}$ and ON-CNN$_{SP}$ CNN accelerator architectures with state-of-the-art CNN accelerator architectures.

Architectures	Year	Tech. (nm)	Freq. (MHz)	Prec. (Bits)	Peak performance (TOPS)	Area (mm^2)	Area Efficiency (TOPS/mm^2)	Power (mW)	Power Efficiency (TOPS/W)	Filter Sizes
Eyeriss [6]	2017	65	250	16	0.0462@AlexNet 0.0214@VGG-16	12.25	0.0037@AlexNet 0.0018@VGG-16	278@AlexNet 236@VGG-16	0.1662@AlexNet 0.0907@VGG-16	1-12 x 1-32
Stripes [12]	2017	45	154	16	1.260@VGG-16 1.188@ResNet-18 1.260@ResNet-50	18.18	0.0693@VGG-16 0.0653@ResNet-18 0.0693@ResNet-50	2732@VGG-16 2238@ResNet-18 2744@ResNet50	0.4612@VGG-16 0.5308@ResNet-18 0.4592@ResNet-50	all
Loom [13]	2018	45	182	16	0.093@VGG-16 0.088@ResNet-18 0.093@ResNet-50	7.79	0.0119@VGG-16 0.0113@ResNet-18 0.0119@ResNet-50	1250@VGG-16 1024@ResNet-18 1255@ResNet50	0.0744@VGG-16 0.0859@ResNet-18 0.0741@ResNet-50	all
FID (A-E) [7]	2018	65	400	16	0.0703@VGG-16	1.77	0.0397@VGG-16	254	0.2768@VGG-16	3 x 3
FID (MA-E) [7]	2018	65	200	16	0.0677@VGG-16	3.5	0.0193@VGG-16	260	0.2604@VGG-16	3 x 3
ZASCAD [8]	2020	65	200	16	0.0725@VGG-16 0.0641@ResNet-18 0.0745@ResNet50	6	0.0121@VGG-16 0.0107@ResNet-18 0.0124@ResNet50	301@VGG-16 253@ResNet-18 248@ResNet50	0.2409@VGG-16 0.2534@ResNet-18 0.3004@ResNet50	all
Res-DNN [9]	2020	45	1200	16	0.278@AlexNet 0.134@VGG-16	2.18@MGE	0.1275 TOPS/MGE @AlexNet 0.0615 TOPS/MGE @VGG-16	-	0.308@AlexNet 0.223@VGG-16	1-12 x 1-32
CARLA [10]	2021	65	200	16	0.0774@VGG-16 0.0754@ResNet50	6.2	0.0125@VGG-16 0.0122@ResNet50	247	0.3134@VGG-16 0.3053@ResNet50	all
BLMAC [11]	2022	65	962	16	0.181@VGG-16	2.5	0.0724@VGG-16	393.6	0.4599@VGG-16	3 x 3
ON-CNN$_{SS}$	2024	45	834	16	2.276@VGG-16 2.144@ResNet-18 2.276@ResNet-50	40.33	0.0564@VGG-16 0.0532@ResNet-18 0.0564@ResNet-50	13325@VGG-16 10918@ResNet-18 13384@ResNet50	0.1708@VGG-16 0.1964@ResNet-18 0.1701@ResNet-50	all
ON-CNN$_{SP}$	2024	45	909	16	2.535@VGG-16 2.389@ResNet-18 2.535@ResNet-50	39.92	0.0635@VGG-16 0.0598@ResNet-18 0.0635@ResNet-50	13193@VGG-16 10809@ResNet-18 13251@ResNet50	0.1922@VGG-16 0.2210@ResNet-18 0.1913@ResNet-50	all

DNN shows better area-efficiency of 0.0615 TOPS/MGE (the area is reported in millions of NAND2 gates (MGE)) in comparison with 0.0564 TOPS/mm^2 of ON-CNN$_{SS}$ for VGG-16. Whereas, ON-CNN$_{SP}$ shows better area-efficiency of 0.0635 TOPS/mm^2 in comparison with Res-DNN. Most architectures show less area-efficiency because in order to become power-efficient, they employ less number of PEs. They improve power-efficiency at the expense of low execution time, peak performance and area-efficiency.

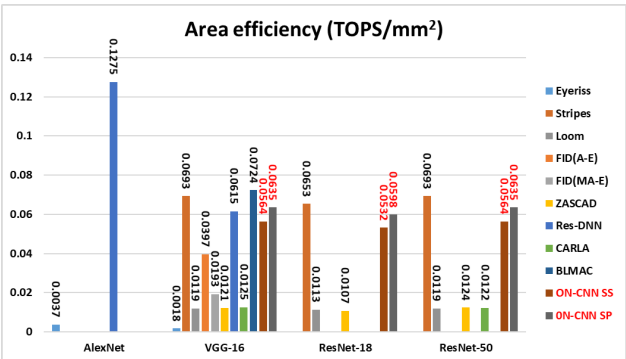

FIGURE 15. Area efficiency comparison with state-of-the-art CNN accelerator architectures. Note that the area efficiency of the architectures is taken from their original papers.

Fig. 16 shows the power-efficiency comparison of ON-CNN$_{SS}$ and ON-CNN$_{SP}$ with state-of-the-art CNN accelerator architectures. It can be seen that power-efficiencies of ON-CNN$_{SS}$ and ON-CNN$_{SP}$ are not better in comparison with state-of-the-art CNN accelerators except Eyeriss and Loom. Even with proposed power saving strategy, ON-CNN$_{SS}$ and ON-CNN$_{SP}$ suffer from huge power consumption

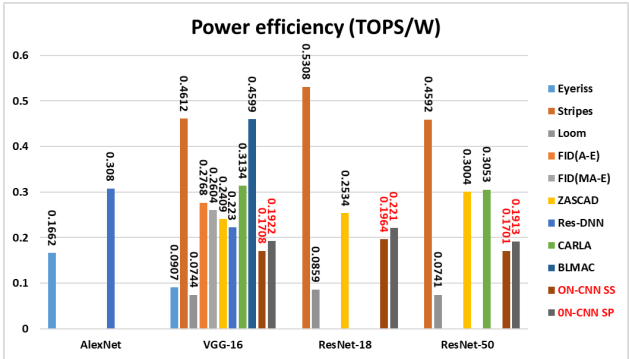

FIGURE 16. Power efficiency comparison with state-of-the-art CNN accelerator architectures. Note that the power efficiency of the architectures is taken from their original papers.

because of big sizes of these architectures. For VGG-16, Res-DNN shows better power-efficiency of 0.223 TOPs/W in comparison with 0.1708 TOPs/W of ON-CNN$_{SS}$ and 0.1922 TOPS/W of ON-CNN$_{SP}$. Res-DNN is based on Eyeriss. It employs ResNS to reduce power consumption. Thus, it achieves better power-efficiency in comparison with Eyeriss. Res-DNN does not provide execution time values for VGG-16. ON-CNN$_{SS}$ and ON-CNN$_{SP}$ have better peak performance than Res-DNN. Thus, we can not say which one is the better architecture. But ON-CNN$_{SP}$ has better peak performance and area-efficiency than Res-DNN. Stripes and BLMAC exhibits highest power efficiency of 0.4612 TOPS/W and 0.4599 TOPS/W in comparison with ON-CNN$_{SS}$ and ON-CNN$_{SP}$ for VGG-16. But they suffer from high execution time and low peak performance in comparison with ON-CNN$_{SS}$ and ON-CNN$_{SP}$.

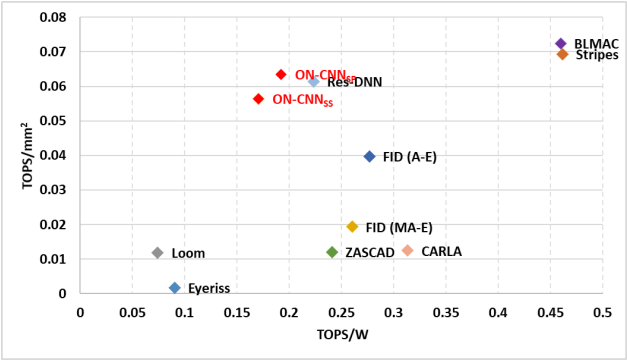

FIGURE 17. Overview of state-of-the-art CNN accelerator architectures.

Fig. 17 compares the area-efficiency ($TOPS/mm^2$) and peak power-efficiency ($TOPS/W$) of the ON-CNN$_{SS}$ and ON-CNN$_{SP}$ accelerators with the state-of-the-art CNN accelerator architectures. Bit-parallel BLMAC and conventional bit-serial Stripes accelerator architectures achieve the right balance between area and power-efficiency. But both suffer from high execution time and low peak performance. Whereas, BLMAC can not process filters of dimension other than 3×3. Thus, it has limited application. Architectures like FID, ZASCAD and CARLA are better suited for applications requiring better power-efficiency. ZASCAD and CARLA can process filter of any dimension. Whereas, architectures like Res-DNN, ON-CNN$_{SS}$ and ON-CNN$_{SP}$ are suitable for applications requiring better area-efficiency. They can also process filter of any dimension. Overall ON-CNN$_{SS}$ and ON-CNN$_{SP}$ show least execution time and highest peak performance in comparison with all other state-of-the-art CNN accelerator architectures.

VI. CONCLUSION

In this work, we have proposed ON-CNN: a CNN accelerator architecture which employs MSDF bit-serial computing offered by online arithmetic to improve latency and throughput. ON-CNN also employs the proposed re-configurable power saving strategy which switches off the idle resources based on the characteristics of convolution layers. ON-CNN$_{SS}$ achieves the execution time of 21.1 ms for VGG-16, 3.6 ms for ResNet-18 and 5.4 ms for ResNet-50. Whereas, ON-CNN$_{SP}$ achieves the execution time of 19 ms for VGG-16, 3.2 ms for ResNet-18 and 4.9 ms for ResNet-50. Depending upon the CNN model, peak throughput of ON-CNN$_{SS}$/ON-CNN$_{SP}$ varies from 2.276 TOPS to 2.535 TOPS. The proposed re-configurable power saving strategy reduces the power consumption by 19.2% for VGG-16, 33.8% for ResNet-18 and 18.9% for ResNet-50 model. Overall, ON-CNN achieves least execution time, highest peak throughput and better area-efficiency in comparison with state-of-the-art CNN accelerator architectures.

ACKNOWLEDGMENT

The EDA tool was supported by the IC Design Education Center (IDEC), South Korea.

REFERENCES

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
[2] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1–9.
[3] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, Jun. 2017, doi: 10.1145/3065386.
[4] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015, *arXiv:1409.1556*.
[5] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 580–587.
[6] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
[7] A. Ardakani, C. Condo, M. Ahmadi, and W. J. Gross, "An architecture to accelerate convolution in deep neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 4, pp. 1349–1362, Apr. 2018.
[8] A. Ardakani, C. Condo, and W. J. Gross, "Fast and efficient convolutional accelerator for edge computing," *IEEE Trans. Comput.*, vol. 69, no. 1, pp. 138–152, Jan. 2020.
[9] N. Samimi, M. Kamal, A. Afzali-Kusha, and M. Pedram, "Res-DNN: A residue number system-based DNN accelerator unit," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 2, pp. 658–671, Feb. 2020.
[10] M. Ahmadi, S. Vakili, and J. M. P. Langlois, "CARLA: A convolution accelerator with a reconfigurable and low-energy architecture," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 8, pp. 3184–3196, Aug. 2021.
[11] S. S. Lee, T. D. Nguyen, P. K. Meher, and S. Y. Park, "Energy-efficient high-speed ASIC implementation of convolutional neural network using novel reduced critical-path design," *IEEE Access*, vol. 10, pp. 34032–34045, 2022.
[12] P. Judd, J. Albericio, and A. Moshovos, "Stripes: Bit-serial deep neural network computing," *IEEE Comput. Archit. Lett.*, vol. 16, no. 1, pp. 80–83, Jan. 2017.
[13] S. Sharify, A. D. Lascorz, K. Siu, P. Judd, and A. Moshovos, "Loom: Exploiting weight and activation precisions to accelerate convolutional neural networks," in *Proc. 55th ACM/ESDA/IEEE Design Autom. Conf. (DAC)*, Jun. 2018, pp. 1–6.
[14] M. D. Ercegovac and T. Lang, *Digital Arithmetic*. San Francisco, CA, USA: Morgan Kaufmann, 2004.
[15] M. Usman, M. D. Ercegovac, and J.-A. Lee, "Low-latency online multiplier with reduced activities and minimized interconnect for inner product arrays," *J. Signal Process. Syst.*, vol. 95, no. 7, pp. 777–796, Jul. 2023.
[16] C. Song, Y. Tang, J. Liu, S. Bian, D. Deng, and H. Li, "MSDF-SGD: Most-significant digit-first stochastic gradient descent for arbitrary-precision training," in *Proc. 33rd Int. Conf. Field-Programmable Log. Appl. (FPL)*, Sep. 2023, pp. 159–165.
[17] S. Bian, H. Li, C. Wang, C. Song, and Y. Tang, "MSBF-LSTM: Most-significant bit-first LSTM accelerators with energy efficiency optimisations," in *Proc. IEEE 31st Annu. Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM)*, May 2023, p. 218.
[18] H. Li, H. Fan, and J. Liang, "Quantum most-significant digit-first addition," in *Proc. 12th Int. Green Sustain. Comput. Conf. (IGSC)*, Oct. 2021, pp. 1–8.
[19] A. M. S. Abdelhadi and L. Shannon, "Revisiting deep learning parallelism: Fine-grained inference engine utilizing online arithmetic," in *Proc. Int. Conf. Field-Programmable Technol. (ICFPT)*, Dec. 2019, pp. 383–386.
[20] M. D. Ercegovac, "On left-to-right arithmetic," in *Proc. 51st Asilomar Conf. Signals, Syst., Comput.*, Oct. 2017, pp. 750–754.
[21] E. Lang, "On-the-fly conversion of redundant into conventional representations," *IEEE Trans. Comput.*, vol. C-36, no. 7, pp. 895–897, Jul. 1987.
[22] R. Galli and A. Tenca, "Design and evaluation of on-line arithmetic for signal processing applications on fpgas," *Proc. SPIE*, vol. 4474, pp. 134–144, Nov. 2001.
[23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2015, pp. 770–778.

[24] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, and O. Temam, "DaDianNao: A machine-learning supercomputer," in *Proc. 47th Annu. IEEE/ACM Int. Symp. Microarchitecture*, Dec. 2014, pp. 609–622.

[25] J. Albericio, A. Delmás, P. Judd, S. Sharify, G. O'Leary, R. Genov, and A. Moshovos, "Bit-pragmatic deep neural network computing," in *Proc. 50th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2017, pp. 382–394.

[26] S. Sharify, A. D. Lascorz, M. Mahmoud, M. Nikolic, K. Siu, D. M. Stuart, Z. Poulos, and A. Moshovos, "Laconic deep learning inference acceleration," in *Proc. ACM/IEEE 46th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2019, pp. 304–317.

[27] P. Darbani, N. Rohbani, H. Beitollahi, and P. Lotfi-Kamran, "RASHT: A partially reconfigurable architecture for efficient implementation of CNNs," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 7, pp. 860–868, Jul. 2022.

[28] P. Darbani, H. Beitollahi, and P. Lotfi-Kamran, "Rei: A reconfigurable interconnection unit for array-based CNN accelerators," *IEEE Trans. Emerg. Topics Comput.*, vol. 11, no. 4, pp. 895–906, Oct./Dec. 2023.

[29] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, Dec. 2015.

[30] A. Stillmaker and B. Baas, "Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm," *Integration*, vol. 58, pp. 74–81, Jun. 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167926017300755

[31] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*. New York, NY, USA: Association for Computing Machinery, Feb. 2015, pp. 161–170, doi: 10.1145/2684746.2689060.

[32] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017, doi: 10.1145/3065386.

MUHAMMAD AKMAL SHAFIQUE (Graduate Student Member, IEEE) received the B.Sc. degree in electrical engineering from the Federal Urdu University of Arts, Science and Technology, Islamabad, Pakistan, in 2015, and the M.Sc. degree in electrical engineering from the University of Engineering and Technology, Taxila, Pakistan, in 2020. He is currently pursuing the Ph.D. degree in computer engineering with Chosun University, Gwangju, South Korea. His research interests include computer arithmetic, convolutional neural network accelerators, network-on-chip architecture, and fault tolerance.

JEONG-A LEE (Life Senior Member, IEEE) received the B.S. degree (Hons.) in computer engineering from Seoul National University, Seoul, South Korea, in 1982, the M.S. degree in computer science from Indiana University Bloomington, Bloomington, IN, USA, in 1985, and the Ph.D. degree in computer science from the University of California at Los Angeles, Los Angeles, CA, USA, in 1990. From 1990 to 1995, she was an Assistant Professor with the Department of Electrical and Computer Engineering, University of Houston, Houston, TX, USA. Since 1995, she has been with Chosun University, Gwangju, South Korea. From 2008 to 2009, she was the Program Director of the ECE Division, National Research Foundation of Korea. Her current research interests include high-performance computer architectures, memory architecture, approximate computing, and reliable computing. She is a member of the National Academy of Engineering, South Korea.